

In this case, each thread sets the shared variable *sacrificer* to its thread ID before attempting to enter the critical section. Even if both threads execute concurrently, *sacrificer* can be set to one of the two thread IDs only, letting the other thread enter the critical section. This avoids the deadlock encountered in our *SimpleLock1* code snippet when both threads execute concurrently. I leave it to the reader to prove that *SimpleLock2* ensures mutual exclusion.

On the other hand, what happens if thread A with ID 0 runs first completely, before thread B with ID 1? In this case, *sacrificer* is set to 0 and thread A waits without being able to acquire the lock. Note that *SimpleLock1* and *SimpleLock2* both suffer from deadlock, but under exactly opposite conditions —while *SimpleLock1* deadlocks if the two threads execute concurrently, *SimpleLock2* deadlocks if thread A runs first completely, before thread B. Can we combine *SimpleLock1* and *SimpleLock2* to get a mutual exclusion algorithm that is deadlock-free?

Takeaway problem

This month's takeaway problem is for readers to come up with a mutual exclusion algorithm for two threads, which is deadlock-free, by combining *SimpleLock1* and *SimpleLock2* algorithms. You need to show that your algorithm is deadlock-free under all conditions.

Learning to code correctly is a never ending exercise. As we learn, we find that there are areas that we have not addressed. Therefore, there are a number of books that all programmers should have on their book shelves. And of course, this list is different for each one of us, since each programmer has a different set of favourite programming books. From this month onwards, I am planning to make a brief mention of one book I found useful in the art of

programming. If you have a favourite programming book that you think is a must-read, please do send me a note with the book's name, and a short description of why you think it is useful. I will include a mention of it in the column; this would help many readers who want to improve their coding skills.

My 'Must-read book for every programmer' this month is, 'The Art of Computer Programming' by Donald Knuth. There are three volumes of this book, and every programmer should definitely read the first volume on fundamental algorithms. While it is not an easy book to read (I personally found that I needed to re-read each chapter in order to grasp all the details), it gives you a solid foundation on which to build your programming skills.

Please do send me your answers to this month's questions, and your recommendations for must-read books for every programmer. If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and wishing you the very best! 

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools. [If you are preparing for computer science programming interviews, you may find it useful to visit Sandya's programming interviews discussion group 'Computer Science Interview Training (India)' on LinkedIn (www.linkedin.com).]

